a

2. Theory Assignment Solutions

Problem 1: The Jacobian and the chain Rule

Task A: According to the Multivariable Chain Rule, the derivative of the composite function $f(g(x))$ is calculated by performing matrix multiplication on their respective Jacobian matrices. specifically: $J_{final} = J_f \times J_g$

Task B: A standard Gradient vector only works for a function that outputs a single scalar number. A neural network layer output a vector of many numbers. To track how every single input impacts every single output we must use a Jacobian matrix. The rows represent the output Varibles and the Columns represent the input variables.

Problem 2: The Hessian and Saddle Points

Task A: Clairaut's Theorem guarantees that for continuously differentiable funcation's

of taking partial derivatives does not matter ($\frac{\partial^2 L}{\partial x \partial y} = \frac{\partial^2 L}{\partial y \partial x}$) Since the off-diagonal elements mirror each other, the Hessian matrix is perfectly symmetric ($H = H^T$).

Task B: Mixed positive and negative eigenvalues define an indefinite matrix, meaning you are at a saddle point. Geometrically, the landscape curves up in one direction and down in another. Gradient Descent stalls here because the gradient approaches zero, and the optimizer struggles to escape the flat plateau without momentum.

Problem 3: The curse of second-order Optimization

Task A: Multiplying by the inverse Hessian (H^{-1}) divides the gradient by the curvature. It ~~mo~~ normalizes the landscape, flattening out steep ravines and allowing the

optimizer to point directly at the geometric center of the minimum accounting for all curvature distortions.

Task B: A Neural Network with N parameters would have an $N \times N$ Hessian matrix. For a modern model with 1 billion parameters, the Hessian would contain 10^{18} elements, storing it is impossible, and calculating the inverse of a matrix take roughly $O(N^3)$ time complexity, which is computationally intractable.